Chapter 8 — Embeddings & Vector Search

8.1 Dense and Sparse Embeddings

Conceptual Deep Dive

An **embedding** maps text (or images, audio, code) to a vector so that semantic closeness becomes geometric closeness. **Dense** embeddings (384–4096 dims) come from encoder transformers trained with contrastive loss (InfoNCE: pull query–positive pairs together, push in-batch negatives apart). **Sparse** embeddings are high-dimensional with mostly zeros: classic **BM25/TF-IDF** (exact term matching, weighted by rarity) and learned sparse models like **SPLADE** (expand terms via a masked-LM head). **Multi-vector / late interaction** (ColBERT) keeps one vector per token and scores by MaxSim — better accuracy, more storage. Dense wins on paraphrase and intent; sparse wins on rare identifiers, names, error codes. Always embed queries and documents with the *same model*, honouring its query/document prefixes (e.g. "query: ", "passage: ").

Cross-Connection Scenarios

Hybrid retrieval and re-ranking (9); semantic caching (4.6); dynamic few-shot (6.1); clustering for data curation (12).

Production Anti-Patterns

- Mixing embedding models in one index.
- Re-embedding the corpus on a model upgrade without versioning the index.
- Using a general model on a niche domain without evaluating retrieval recall.

Minimal Working Code

```
import math
from collections import Counter

def bm25_scores(query: str, docs: list[str], k1=1.5, b=0.75):
    tok = [d.lower().split() for d in docs]
    avg = sum(map(len, tok)) / len(tok)
    n = len(docs)
    df = Counter(t for d in tok for t in set(d))
    out = []
    for d in tok:
        tf, s = Counter(d), 0.0
        for q in query.lower().split():
            if q not in tf:
                continue
            idf = math.log((n - df[q] + 0.5) / (df[q] + 0.5) + 1)
            s += idf * tf[q] * (k1 + 1) / (tf[q] + k1 * (1 - b + b * len(d) / avg))
        out.append(s)
    return out

print(bm25_scores("error 503", ["http error 503 timeout", "happy path", &qu
ot;503 service"]))
```

8.2 Similarity Metrics and Dimensionality

Conceptual Deep Dive

Cosine similarity = a·b / (||a||||b||) measures angle; for L2-normalised vectors it equals the dot product, and Euclidean distance is a monotone function of it (||a–b||^2 = 2 – 2cos). Use the metric the model was trained for. **Matryoshka Representation Learning (MRL)** trains embeddings so the first k dimensions are themselves useful: truncate (and re-normalise) to cut storage and latency with small accuracy loss. **Dimensionality reduction**: PCA, random projection, and quantisation (scalar int8, binary with Hamming distance, product quantisation) shrink memory 4–32×; rescore top candidates with full-precision vectors. Distance concentration in high dimensions makes absolute thresholds fragile; calibrate on labelled pairs.

Cross-Connection Scenarios

Index memory planning (8.3); re-ranking compensates for compression (9.4); evaluation of retrieval (10).

Production Anti-Patterns

- Forgetting to normalise when the index uses dot product.
- A single global similarity threshold across query types.
- Truncating non-Matryoshka embeddings.

Minimal Working Code

```
import numpy as np

def normalize(x: np.ndarray) -> np.ndarray:
    return x / np.linalg.norm(x, axis=-1, keepdims=True)

def matryoshka_truncate(x: np.ndarray, k: int) -> np.ndarray:
    return normalize(x[..., :k])

def binary_quantize(x: np.ndarray) -> np.ndarray:
    return np.packbits(x > 0, axis=-1)

def hamming(a: np.ndarray, b: np.ndarray) -> int:
    return int(np.unpackbits(a ^ b).sum())

v = np.random.default_rng(1).standard_normal((2, 1024)).astype(np.float32)
print(float(normalize(v)[0] @ normalize(v)[1]), hamming(binary_quantize(v[0]), binary_quantize(v[1])))
```

8.3 Approximate Nearest Neighbour Indexes

Conceptual Deep Dive

Exact kNN is O(N·d) per query. ANN trades a bit of recall for speed. **HNSW**: a multi-layer proximity graph; search greedily descends from sparse upper layers to the dense bottom layer. Parameters: M (links per node), efConstruction, efSearch (higher → better recall, slower). Memory-hungry (graph + vectors in RAM). **IVF**: k-means partition into nlist cells, probe nprobe nearest cells; cheaper in memory, pair with **PQ** (product quantisation) for compression (IVF-PQ). **DiskANN/Vamana**: SSD-resident graph for billion-scale. **ScaNN**: anisotropic quantisation. Memory estimate: N × d × bytes (+ ~M×8bytes/node for HNSW links). Filtering is the hard part: pre-filter (restrict candidates first), post-filter (may return too few), or filtered-HNSW traversal.

Cross-Connection Scenarios

Capacity planning (13); hybrid filters and multi-tenancy (8.5); recall measurement uses eval methods (10).

Production Anti-Patterns

- Tuning efSearch without measuring recall against brute force.
- Post-filtering with highly selective filters (empty results).
- Rebuilding the whole index for every small update.

Minimal Working Code

```
import numpy as np

class IVFIndex:
    def __init__(self, vectors: np.ndarray, nlist: int = 16, iters: int = 5, seed=0):
        rng = np.random.default_rng(seed)
        self.v = vectors
        self.c = vectors[rng.choice(len(vectors), nlist, replace=False)].copy()
        for _ in range(iters):
            assign = np.argmax(vectors @ self.c.T, axis=1)
            for j in range(nlist):
                m = vectors[assign == j]
                if len(m):
                    self.c[j] = m.mean(0)
        self.assign = np.argmax(vectors @ self.c.T, axis=1)

    def search(self, q: np.ndarray, k=5, nprobe=3):
        cells = np.argsort(-(self.c @ q))[:nprobe]
        ids = np.where(np.isin(self.assign, cells))[0]
        scores = self.v[ids] @ q
        top = np.argsort(-scores)[:k]
        return ids[top], scores[top]

X = np.random.default_rng(2).standard_normal((2000, 32)).astype(np.float32)
X /= np.linalg.norm(X, axis=1, keepdims=True)
print(IVFIndex(X).search(X[0])[0])
```

8.4 Vector Databases: Qdrant, Milvus, Pinecone, pgvector

Conceptual Deep Dive

System	Model	Strengths	Watch for
Qdrant	Open source (Rust), self-host or cloud	Rich payload filtering, HNSW, quantisation, sparse+dense hybrid	Ops for clustering at scale
Milvus	Open source, distributed	Billion-scale, many index types (HNSW, IVF, DiskANN, GPU)	Operational complexity
Pinecone	Managed, serverless	Zero ops, namespaces, easy scaling	Cost, vendor lock-in, less tunable
pgvector	Postgres extension	Transactions, joins, one datastore, HNSW/IVFFlat	Scaling beyond tens of millions of vectors; tuning
Others	Weaviate, Chroma, FAISS (library), Elasticsearch/OpenSearch, Vespa		

Selection heuristics: already on Postgres and <~10M vectors → pgvector; heavy filtering and self-hosted → Qdrant; very large scale → Milvus/Vespa; no ops team → managed. Evaluate with your data: recall@k, p95 latency, filtered-query latency, ingest rate, cost.

Cross-Connection Scenarios

RAG (9); multi-tenant security (14); deployment (13).

Production Anti-Patterns

- Choosing a vector DB from a benchmark blog instead of your workload.
- Storing only vectors without the source text/metadata needed to cite.
- No backup/snapshot strategy.

Minimal Working Code

```
# Qdrant (pip install qdrant-client)
from qdrant_client import QdrantClient, models

client = QdrantClient(":memory:")
client.create_collection(
    "docs",
    vectors_config=models.VectorParams(size=4, distance=models.Distance.COSINE),
)
client.upsert("docs", points=[
    models.PointStruct(id=1, vector=[0.1, 0.9, 0.0, 0.0], payload={"tenant": "a", &qu
ot;text": "hello"}),
    models.PointStruct(id=2, vector=[0.9, 0.1, 0.0, 0.0], payload={"tenant": "b", &qu
ot;text": "world"}),
])
hits = client.query_points(
    "docs", query=[0.1, 0.8, 0.1, 0.0], limit=1,
    query_filter=models.Filter(must=[models.FieldCondition(key="tenant", match=models.MatchValu
e(value="a"))]),
).points
print(hits[0].payload)

# pgvector:  CREATE EXTENSION vector;
#   CREATE TABLE items(id bigserial, emb vector(384), body text);
#   CREATE INDEX ON items USING hnsw (emb vector_cosine_ops);
#   SELECT body FROM items ORDER BY emb <=> $1 LIMIT 5;
```

8.5 Hybrid Search and Metadata Filtering

Conceptual Deep Dive

Hybrid combines dense (semantic) and sparse (lexical) retrieval, then fuses ranks. **Reciprocal Rank Fusion**: score(d) = Σ 1/(k + rank_i(d)), with k≈60 — robust because it ignores incompatible raw score scales. Alternatives: weighted score normalisation (min–max/z-score), learned fusion. Metadata filters (tenant, date, ACL, language) enforce access control and relevance; always apply ACL filters **inside** the retrieval query, never after. Multi-tenancy: per-tenant collections, namespaces, or payload filters with a partition key.

Cross-Connection Scenarios

Fusion is revisited in advanced RAG (9.3); ACL filtering is a security control (14).

Production Anti-Patterns

- Filtering ACLs in application code after retrieval (leaks via ranking and top-k starvation).
- Weighted sums of raw BM25 and cosine scores.
- Ignoring recency for time-sensitive corpora.

Minimal Working Code

```
from collections import defaultdict

def rrf(rank_lists: list[list[str]], k: int = 60, top: int = 10) -> list[str]:
    scores = defaultdict(float)
    for ranks in rank_lists:
        for r, doc in enumerate(ranks, start=1):
```

```
            scores[doc] += 1.0 / (k + r)
    return sorted(scores, key=scores.get, reverse=True)[:top]

print(rrf([["a", "b", "c"], ["c", "a", "d"]])
)
```

8.6 Embedding Operations: Batching, Versioning, Drift

Conceptual Deep Dive

Embed in batches with concurrency limits (4.3), cache by content hash, store embedding_model and embedding_version per vector, and do blue/green index migrations (build new index, shadow-query, compare recall, flip alias). Monitor query-embedding drift (average similarity to nearest neighbours trending down signals domain shift). Fine-tune embeddings with hard negatives mined from logs when off-the-shelf recall is the bottleneck.

Cross-Connection Scenarios

Eval datasets (10); CI regressions on retrieval recall (15); model fine-tuning methods (12).

Production Anti-Patterns

- Embedding on every query for static content (cache).
- Silent model-version change at provider.
- No dedupe → near-duplicate chunks crowd out results.

Minimal Working Code

```
import hashlib
import json

class EmbedCache:
    def __init__(self, embed_fn, model_id: str):
        self.fn, self.model, self.store = embed_fn, model_id, {}

    def _k(self, text):
        return hashlib.sha256(f"{self.model}:{text}".encode()).hexdigest()

    def embed(self, texts: list[str]):
        missing = [t for t in texts if self._k(t) not in self.store]
        if missing:
            for t, v in zip(missing, self.fn(missing)):
                self.store[self._k(t)] = v
        return [self.store[self._k(t)] for t in texts]

c = EmbedCache(lambda ts: [[float(len(t))] for t in ts], "toy-v1")
print(c.embed(["a", "bb", "a"]))
```

8.7 Choosing and Fine-Tuning Embedding Models

Conceptual Deep Dive

Selection criteria: retrieval quality on **your** data, max input length, dimension (storage and latency), multilingual coverage, licence, hosting (API versus local), and throughput. Procedure: build 100+ query-to-relevant-document pairs, embed with 3–5 candidates, compare recall@k and MRR, then check cost. Fine-tune when domain vocabulary is specialised: collect (query, positive) pairs from logs or synthetic generation, mine **hard negatives** (top-ranked non-relevant results), train with contrastive loss (MultipleNegativesRankingLoss), and validate on held-out queries. Consider adapters or a linear transform on top of frozen embeddings as a cheap alternative. Edge cases: synthetic queries that are too easy, false negatives among mined hard negatives, and embedding drift after a vendor model update.

Cross-Connection Scenarios

InfoNCE objective (5.8), retrieval metrics (10.2), index versioning (8.6).

Production Anti-Patterns

- Picking the top MTEB model without a domain test.
- Training on queries that duplicate the test set.
- Mining hard negatives without human spot-checks.

Minimal Working Code

```
def evaluate_embedder(embed, queries, corpus, relevant, k=5):
    """queries: {qid: text}; corpus: {did: text}; relevant: {qid: set(did)}.""&q
uot;
    import numpy as np
    dids = list(corpus)
    D = np.array(embed([corpus[d] for d in dids]), dtype=np.float32)
    D /= np.linalg.norm(D, axis=1, keepdims=True)
    hits = 0.0
    for qid, text in queries.items():
        q = np.array(embed([text])[0], dtype=np.float32)
        q /= np.linalg.norm(q)
        top = [dids[i] for i in np.argsort(-(D @ q))[:k]]
        hits += len(set(top) & relevant[qid]) / len(relevant[qid])
    return hits / len(queries)

toy_embed = lambda texts: [[len(t), t.count("a"), 1.0] for t in texts]
print(evaluate_embedder(toy_embed, {"q1": "aaa"}, {"d1": "aaaa",
"d2": "b"}, {"q1": {"d1"}}, k=1))
```

```
# pip install sentence-transformers
from sentence_transformers import InputExample, SentenceTransformer, losses
from torch.utils.data import DataLoader

def finetune(pairs, base="BAAI/bge-small-en-v1.5", epochs=1):
    model = SentenceTransformer(base)
    examples = [InputExample(texts=[q, pos]) for q, pos in pairs]
    loader = DataLoader(examples, shuffle=True, batch_size=32)
    loss = losses.MultipleNegativesRankingLoss(model)
    model.fit(train_objectives=[(loader, loss)], epochs=epochs, warmup_steps=50)
    return model
```

8.8 Index Operations: Updates, Deletions, Sharding and Filters at Scale

Conceptual Deep Dive

Real indexes change constantly. HNSW supports inserts but deletions leave tombstones that degrade the graph until compaction or rebuild; IVF needs periodic re-training of centroids when the data distribution shifts. Strategies: write to a small mutable segment plus large immutable segments (LSM-style), merge in the background, and soft-delete with a filter for immediate effect. **Sharding**: by tenant (isolation) or by hash (balance); query all shards and merge top-k. **Replication** for availability. **Filtered search** at high selectivity may require payload indexes or brute-force fallback. **Quantisation + rescoring** reduces RAM: search compressed vectors, rescore top-N with originals on disk. Edge cases: delete-by-document when documents map to many chunks, orphan vectors, concurrent upsert races, and snapshots during writes.

Cross-Connection Scenarios

Ingestion sync (17.2), multi-tenancy (8.5, 4.7), capacity planning (13).

Production Anti-Patterns

- Deleting a document but leaving its chunks searchable.
- Never compacting an index with heavy churn.
- Single-node index with no snapshot.

Minimal Working Code

```
import numpy as np

class TwoTierIndex:
    """Small mutable buffer + big immutable main segment; deletions are soft.""&
quot;
    def __init__(self, main_vecs: np.ndarray, main_ids: list[str]):
        self.main, self.main_ids = main_vecs, main_ids
        self.buf, self.buf_ids, self.deleted = [], [], set()

    def add(self, vec, id_):
        self.buf.append(vec)
        self.buf_ids.append(id_)

    def delete(self, id_):
        self.deleted.add(id_)

    def search(self, q, k=3):
        vecs = np.vstack([self.main] + ([np.array(self.buf)] if self.buf else []))
        ids = self.main_ids + self.buf_ids
        scores = vecs @ q
        out = [(ids[i], float(scores[i])) for i in np.argsort(-scores) if ids[i] not in self.deleted]
        return out[:k]

idx = TwoTierIndex(np.eye(3, dtype=np.float32), ["a", "b", "c"])
idx.add(np.array([0.9, 0.1, 0], dtype=np.float32), "d")
idx.delete("a")
print(idx.search(np.array([1, 0, 0], dtype=np.float32)))
```

```
def delete_document(store: dict, doc_id: str) -> int:
    """Remove every chunk that belongs to doc_id; store maps chunk_id -> {'doc': ..., .
..}."""
    victims = [cid for cid, meta in store.items() if meta["doc"] == doc_id]
    for cid in victims:
        del store[cid]
    return len(victims)

s = {"c1": {"doc": "A"}, "c2": {"doc": "A"},
"c3": {"doc": "B"}}
print(delete_document(s, "A"), list(s))
```

8.9 Late Interaction (ColBERT) and Learned Sparse Retrieval (SPLADE)

Conceptual Deep Dive

Single-vector bi-encoders compress a whole passage into one vector, losing detail. **ColBERT-style late interaction** keeps one vector per token and scores a query-document pair by **MaxSim**: for each query token take its maximum similarity over document tokens, then sum. It captures fine-grained matches and often beats single-vector retrieval out of domain, at the cost of storing many vectors per document (mitigated by residual compression and token pruning; **PLAID**-style engines make it fast). **Learned sparse retrieval (SPLADE)** uses a masked-language-model head to produce a sparse weighted bag of vocabulary terms, including *expansion* terms not present in the text, searchable with an inverted index, combining lexical precision with semantic expansion and good interpretability. Choosing: single dense vector for simplicity and speed, hybrid with BM25 for most production systems, late interaction when top-end quality matters and storage is acceptable, SPLADE when you already operate

inverted-index infrastructure. Edge cases: index size blow-up, out-of-vocabulary identifiers still favouring BM25, and multi-vector support varying across vector databases.

Cross-Connection Scenarios

Hybrid search (8.5), reranking (9.4), embedding training objectives (5.8), index operations (8.8).

Production Anti-Patterns

- Adopting multi-vector retrieval without measuring the storage multiplier.
- Dropping BM25 entirely for corpora full of part numbers and error codes.
- Treating learned sparse weights as calibrated probabilities.

Minimal Working Code

```
import numpy as np

def maxsim(query_vecs: np.ndarray, doc_vecs: np.ndarray) -> float:
    """query_vecs: (q, d), doc_vecs: (t, d); both L2-normalised."""
    sim = query_vecs @ doc_vecs.T          # (q, t)
    return float(sim.max(axis=1).sum())

def normalize(x):
    return x / np.linalg.norm(x, axis=-1, keepdims=True)

rng = np.random.default_rng(0)
q = normalize(rng.standard_normal((4, 16)))
d_relevant = normalize(np.vstack([q + 0.05 * rng.standard_normal(q.shape), rng.standard_normal((6, 16))])
)
d_other = normalize(rng.standard_normal((10, 16)))
print(maxsim(q, d_relevant) > maxsim(q, d_other))
```

```
def sparse_dot(q: dict[str, float], d: dict[str, float]) -> float:
    small, big = (q, d) if len(q) < len(d) else (d, q)
    return sum(w * big.get(t, 0.0) for t, w in small.items())

def invert(docs: dict[str, dict[str, float]]):
    inv: dict[str, list[tuple[str, float]]] = {}
    for did, terms in docs.items():
        for t, w in terms.items():
            inv.setdefault(t, []).append((did, w))
    return inv

def search(inv, q: dict[str, float], k=3):
    scores: dict[str, float] = {}
    for t, qw in q.items():
        for did, dw in inv.get(t, []):
            scores[did] = scores.get(did, 0.0) + qw * dw
    return sorted(scores.items(), key=lambda x: -x[1])[:k]

docs = {"d1": {"refund": 1.2, "return": 0.7}, "d2": {"shippi
ng": 1.0}}   # 'return' is an expansion term
print(search(invert(docs), {"return": 1.0, "refund": 0.5}))
```

8.10 Measuring and Tuning ANN Recall, Latency and Cost

Conceptual Deep Dive

An ANN index is only as good as its measured **recall against exact search**. Procedure: sample 1,000+ real queries, compute ground-truth top-k with brute force, then sweep index parameters (efSearch, nprobe, quantisation level) and record recall@k, p50/p95 latency and memory. Plot the recall-latency curve and choose the knee that meets your SLO. Evaluate under

concurrency (QPS saturation), with **filters** applied (recall often drops sharply for selective filters), and after **updates** (recall can degrade with churn). Cost model: memory = N x d x bytes x overhead; pair with replica count for QPS and cross-AZ egress. Quantisation (scalar/binary/PQ) with oversampling and rescoring typically recovers most recall. Edge cases: queries that are out-of-distribution relative to the indexed data, duplicated vectors producing tie-order differences, and benchmark datasets whose distribution differs from yours.

Cross-Connection Scenarios

ANN internals (8.3), index operations (8.8), capacity planning (13.7), retrieval metrics (10.2).

Production Anti-Patterns

- Setting `efSearch` from a blog post without a recall measurement.
- Reporting average recall that hides selective-filter queries.
- Benchmarking at one QPS and assuming linear scaling.

Minimal Working Code

```
import numpy as np

def recall_at_k(approx_ids: list[list[int]], exact_ids: list[list[int]], k: int) -> float:
    hit = sum(len(set(a[:k]) & set(e[:k])) for a, e in zip(approx_ids, exact_ids))
    return hit / (k * len(exact_ids))

def exact_topk(queries: np.ndarray, corpus: np.ndarray, k: int) -> list[list[int]]:
    return [list(np.argsort(-(corpus @ q))[:k]) for q in queries]

rng = np.random.default_rng(1)
C = rng.standard_normal((2000, 32)).astype(np.float32)
Q = rng.standard_normal((20, 32)).astype(np.float32)
gt = exact_topk(Q, C, 10)
noisy = [[i for i in row[:8]] + [int(rng.integers(0, 2000)) for _ in range(2)] for row in gt]
print(round(recall_at_k(noisy, gt, 10), 3))
```

```
import time

def sweep(search_fn, params: list[dict], queries, exact, k=10):
    """search_fn(query, **params)->ids. Returns recall/latency per setting.""
"
    rows = []
    for p in params:
        t0, out = time.perf_counter(), []
        lat = []
        for q in queries:
            s = time.perf_counter()
            out.append(search_fn(q, **p))
            lat.append(time.perf_counter() - s)
        lat.sort()
        rows.append({"params": p, "recall": recall_at_k(out, exact, k),
                     "p50_ms": 1e3 * lat[len(lat) // 2], "p95_ms": 1e3 * lat[int(.95
* len(lat))]})
    return rows
```